



# In-Memory Processing

Grundlagen Verteilter Systeme

Michael Schöttner

SS 2024

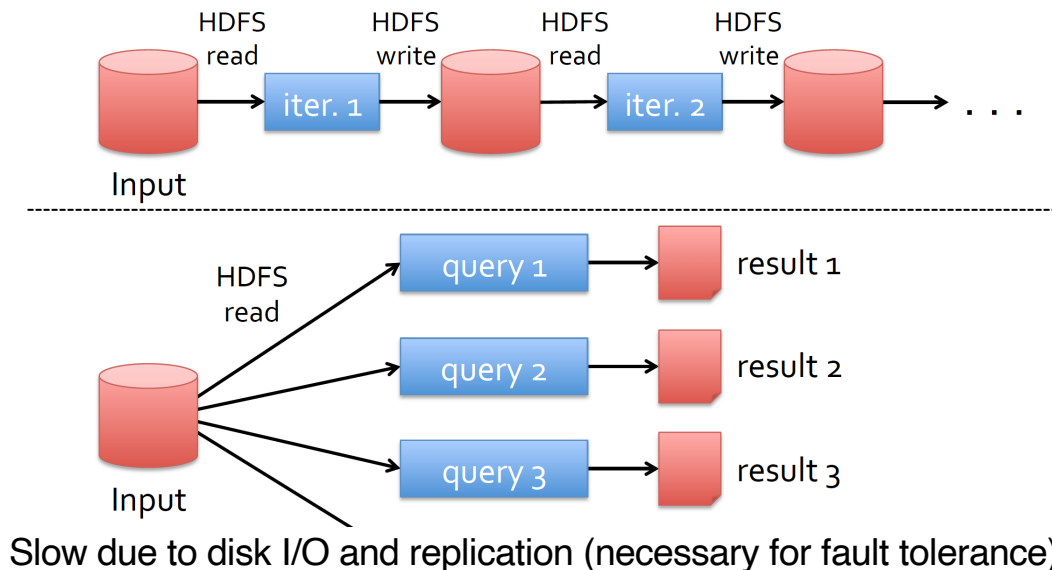
- Motivation
- Overview
- Spark Programming Interface
- Representing RDDs
- Implementation
- Higher-Level Abstractions

# Motivation: limited API of MapReduce

- MapReduce greatly simplified “big data” analysis on large clusters
  - But has a high latency for getting results
  - And programming in MapReduce is difficult
  - Not a novel idea - it represents a specific implementation of well known techniques developed many years ago
- But as soon as it got popular, users wanted more
  - e.g. iterative machine learning & graph processing, interactive ad-hoc queries
- Many specialized frameworks have been proposed

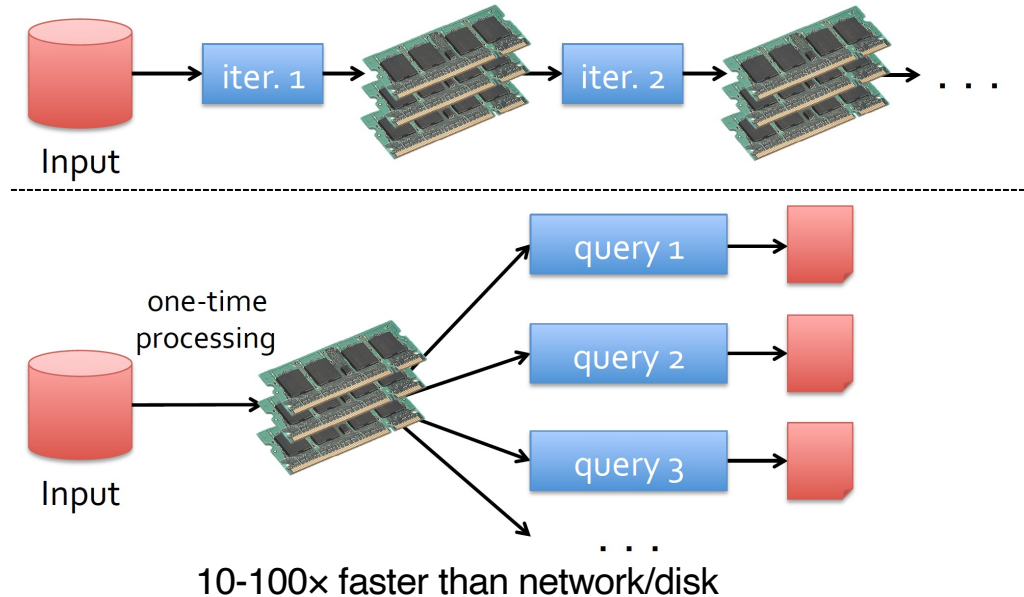
# Motivation: data sharing is slow

- Complex apps and interactive queries both need one thing that MapReduce lacks  
→ efficient primitives for data sharing
- In MapReduce, the only way to share data across jobs is stable storage → slow!
- Examples:



# Motivation: keeping data in memory

- Avoid costly I/O by keeping intermediate results in memory



- Motivation
- Overview
- Spark Programming Interface
- Representing RDDs
- Implementation

- How to design a distributed memory abstraction that is both fault-tolerant and efficient?
- Existing storage abstractions have interfaces based on **fine-grained updates** to **mutable** state, e.g.
  - Distributed caches
  - In-memory databases
  - In-memory key-value stores
- Requires replicating data or logs across nodes for fault tolerance
  - Costly for data-intensive apps
  - Remote logging is 10-100x slower than local memory write

# Solution: RDDs proposed by Spark

- Initially started at UC Berkeley in 2009; written in Scala
- Fast and general framework for large-scale data processing and analysis:
  - 10x (on disk) - 100x (in-memory) faster
- **RDDs = Resilient Distributed Datasets**
  - Resilient – if data in memory is lost, it can be recomputed
  - Distributed – data is stored in memory across the cluster
  - Dataset – initial data can come from a file or created programmatically
- Spark is a restricted form of distributed shared memory
  - Immutable, partitioned collections of records
  - Can only be built through coarse-grained deterministic transformations (map, filter, join, ...)



- High-level language for the Java Virtual Machine (JVM)
  - Object oriented + functional programming
- Comparable in speed to Java
- Often shorter code
- Interoperable with Java
- Fewer errors:
  - No Null Pointer problems
- More flexibility:
  - As many public classes per source file as you want
  - Operator overloading



- RDDs track the graph of operations used to construct them, called **lineage**
- Efficient fault recovery using lineage
  - Log one operation to apply to many elements
  - Recompute lost data on failure from other datasets
  - No cost if nothing fails because no replication is needed
- The larger the lineage, more re-computation is needed, and thus recovery from failure will be longer.
- Therefore, RDDs prefer operations that touch a large number of records at the same time over long lineages

- Spark API is provided in Scala language
  - Each dataset is represented as an object
  - Transformations are invoked using methods on these objects
- Provides:
  - Resilient distributed datasets (RDDs)
  - Operations on RDDs:
    - Transformations: build new RDDs
    - Actions: compute and output results
  - Control of each RDD's partitioning (layout across nodes, see later) and persistence (in RAM or on disk)

# Example Log Mining

- Load error messages from a large log into memory, then interactively search for various patterns

```
lines      = spark.textFile("hdfs://... ")  
errors     = lines.filter(_.startsWith("ERROR"))  
messages  = errors.map(_.split('\t')(2))  
messages.persist()
```

Base RDD, textfile in HDFS

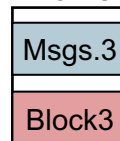
Transformed RDD

Keep message RDDs in memory (to be re-used)

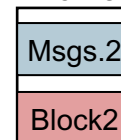
Transformed RDD  
Tab-separated fields, get field no. 2 = message

Master

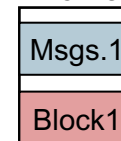
Worker



Worker



Worker



- Nothing happens so far on the cluster

Msgs. RDDs

HDFS block

- Load error messages from a large log into memory, then interactively search for various patterns

```
lines      = spark.textFile("hdfs://...")
errors     = lines.filter(_.startsWith("ERROR"))
messages  = errors.map(_.split('\t')(2))
messages.persist()
```

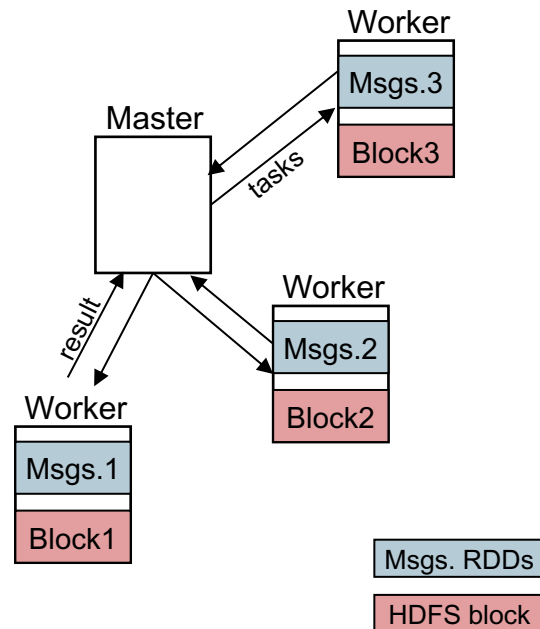
**Action:** count  
number of lines  
containing "foo"

```
messages.filter(_.contains("foo")).count()
```

- An action fires the execution

- Master looks where the data is stored and requests workers to do the work
- RDDs are kept in memory (persist)

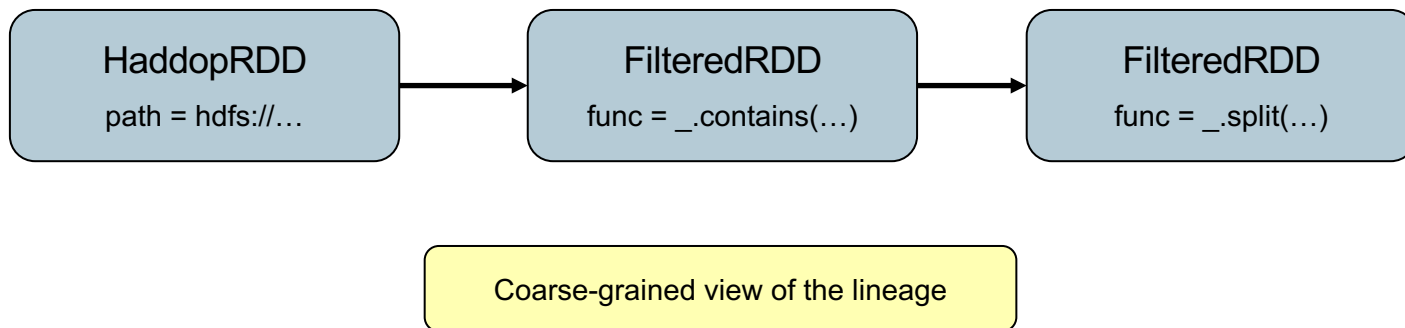
**Result:** full text search in 1 TB:  
Spark: 5-7s  
MapReduce: 170s  
(on a 100 node cluster)



# Example Fault Recovery

- RDDs track the graph of transformations that built them (their lineage) to rebuild lost data

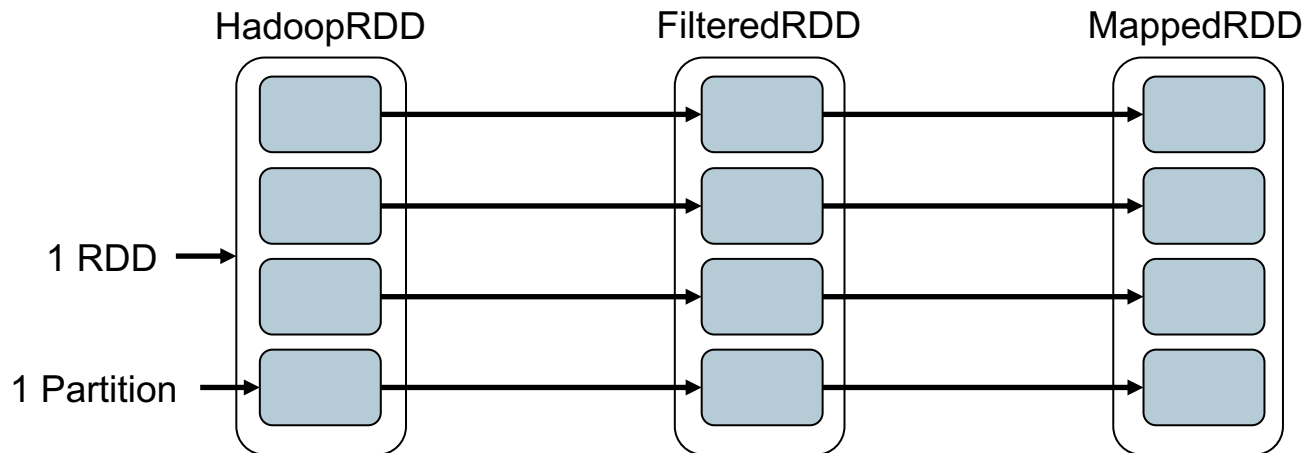
```
messages = textFile(...).filter(_.contains("error"))  
.map(_.split('\t')(2))
```



# Example Fault Recovery

- RDDs track the graph of transformations that built them (their lineage) to rebuild lost data **partitions** (partitions of one RDD are spread over servers)

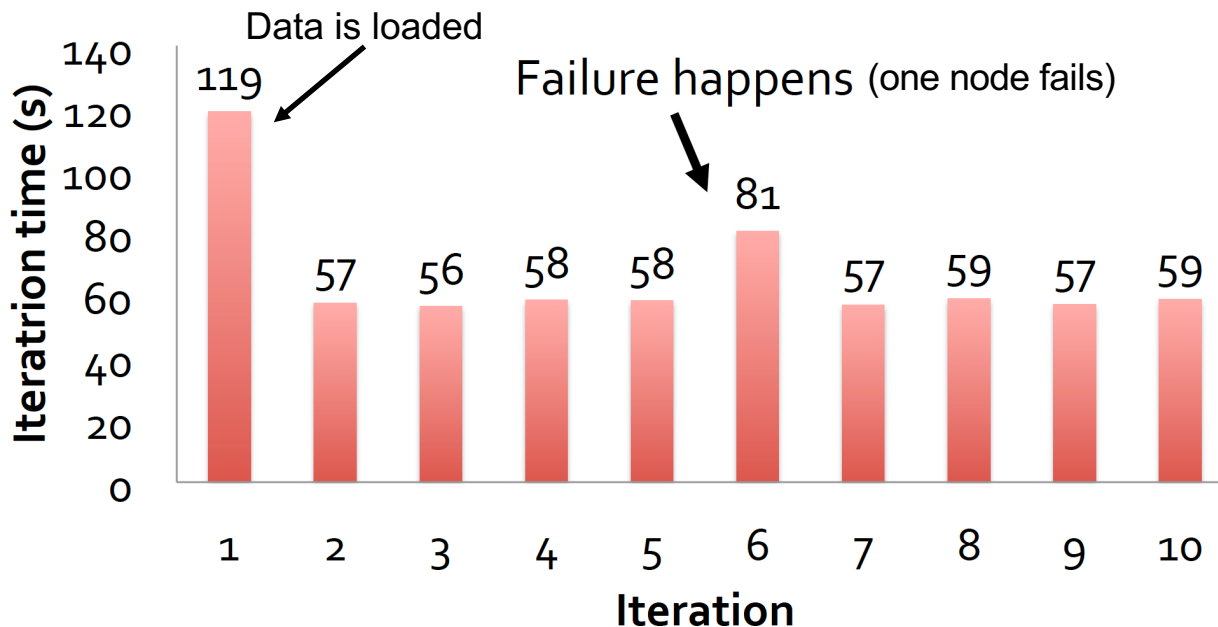
```
messages = textFile(...).filter(_.contains("error"))  
                        .map(_.split('\t')(2))
```



This is the lineage graph  
which is based on  
partition dependencies

# Example Fault Recovery Results

- Iterative machine-learning algorithm (clustering using k means), executed on 75 nodes



- To understand the benefits of RDDs as a distributed memory abstraction, we compare them against distributed shared memory (DSM)

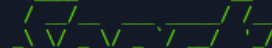
| Aspect                     | RDDs                                        | Distr. Shared Mem.                        |
|----------------------------|---------------------------------------------|-------------------------------------------|
| Reads                      | Coarse- or fine-grained                     | Fine-grained                              |
| Writes                     | Coarse-grained                              | Fine-grained                              |
| Consistency                | Trivial (immutable)                         | Up to app / runtime                       |
| Fault recovery             | Fine-grained and low-overhead using lineage | Requires checkpoints and program rollback |
| Straggler mitigation       | Possible using backup tasks                 | Difficult                                 |
| Work placement             | Automatic based on data locality            | Up to app (runtimes aim for transparency) |
| Behavior if not enough RAM | Similar to existing data flow systems       | Poor performance (swapping?)              |

- Motivation
- Overview
- Spark Programming Interface
- Representing RDDs
- Implementation

## Scala Shell: spark-shell

- Useful for learning or data exploration
- Scala or Python

```
Welcome to  

 version 3.1.2  

Using Scala version 2.12.10 (OpenJDK 64-Bit Server VM, Java 1.8.0_292)  

Type in expressions to have them evaluated.  

Type :help for more information.  

scala>
```

## Python Shell: pyspark

- For large scale data processing
- Python, Java or Scala

```
Welcome to  

      /_/_/_/_/_/_/_/_/_/_\  

     / _V_ V_ V_ V_ V_ V_\  

    / _/  /  /  /  /  /  \  

   / _/  /  /  /  /  /  \  

  / _/  /  /  /  /  /  \  

 / _/  /  /  /  /  /  \  

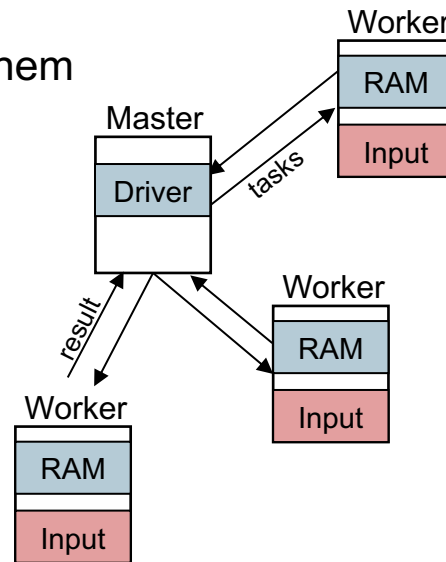
/_/  /  /  /  /  /  \  

version 3.1.2
```

Using Python version 3.9.5 (default, May 11 2021 08:20:37)  
Spark context Web UI available at http://192.168.104.213:4040  
Spark context available as 'sc' (master = local[\*], app id = local-1637568271531).  
SparkSession available as 'spark'.  
>>>

# Running Spark applications

- Developers write a driver program that is started on the master
- The driver defines one or more RDDs and invokes actions on them
- Spark code on the driver also tracks the RDDs' lineage
- The workers are long-lived processes that can store RDD partitions in RAM across operations

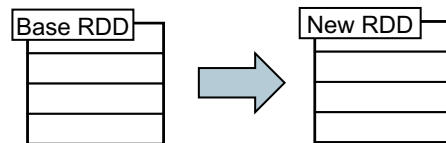


- Data structures that either
  - Point to a direct data source (e.g. HDFS, HBase)
  - Apply some transformations to their parent RDD(s) to generate new RDDs
- Can hold any type of data
  - Primitive types
    - integers, characters, booleans, strings, etc.
  - Sequence types
    - lists, arrays, tuples etc.
  - Scala/Java objects
    - If serializable; including user defined
- Pair RDDs types:
  - Some operations, such as join, are only available on RDDs of key-value pairs (see later)
  - Keys and values can be of any type

## ■ Two types of operations

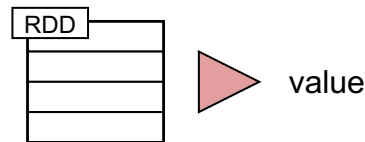
### ■ Transformations:

- Are lazy operations that define a new RDD based on the current one
- Examples: `filter`, `map`



### ■ Actions

- Launch a computation to return a value to the program or write data to external storage.
- Examples: `count`, `take(n)`, `reduce`



## ■ Transformations aren't applied to an RDD until an action is called!

- Users provide arguments to RDD operations by passing closures (functions)
- Scala represents closures as Java objects
- These objects can be serialized and transferred to other workers
  - It is cheap to send a small function object
  - Than transferring large amounts of data

# Example: actions `count` and `first`

Welcome to



Using Scala version 2.12.10 (OpenJDK 64-Bit Server VM, Java 1.8.0\_265)

Type in expressions to have them evaluated.

Type :help for more information.

```
scala> val textFile = sc.textFile("README.md")
```

```
textFile: org.apache.spark.rdd.RDD[String] = README.md MapPartitionsRDD[1] at textFile at <console>:24
```

```
scala> textFile.count()
```

```
res0: Long = 108
```

```
scala> textFile.first()
```

```
res1: String = # Apache Spark
```

```
scala> |
```

## Example: more actions

- `collect()`: returns all the elements of the RDD as an array to the driver, typically after at least one `filter` operation has been applied
- `countByKey()`: for pair RDDs returns `(K, Int)` pairs with the count of each key `K`
- `take(n)`: returns an array with first `n` elements
- `saveAsTextFile()`: writes the elements of the RDD as a text file to HDFS or local filesystem
- ...

# Example: transformation `map`

- Return a new RDD by applying a function to each element of this RDD

- Program:

```
var a = Array(3,4,5)
var x = sc.parallelize( a )
var y = x.map(z => (z, z*2) )
x.collect().foreach(println)
y.collect().foreach(println)
```

The elements of the Array collection `a` are copied to form the distributed dataset `x` that can be operated on in parallel. In the Spark shell, a special interpreter-aware SparkContext is already created for you, in the variable called `sc`.

Apply closure to each  
element in set `x`

Print all elements in set  
`x` and `y`

Retrieve all elements of  
a RDD from all nodes

- Source-RDD `x`: 3, 4, 5

- Map-RDD `y`: (3, 6), (4, 8), (5, 10)

# Example: transformation flatMap

- Return a new RDD by applying a function to each element of the RDD, similar to map, but flatMap can return for each element 0, 1 or several elements

- Program:

```
def g(v:Int) = List(v, v*2)
```

Function to be applied

```
var list = List(1,2,3,4,5)
```

Define a list

```
var list2 = list.flatMap(x => g(x) )
```

Apply g to each element of list

```
list2.foreach( (el:Int) => print(el+ " "))
```

Print each element

- Source-RDD list: 1, 2, 3, 4, 5
- flatMap-RDD list2: 1 2 2 4 3 6 4 8 5 10

# Example: transformation `reduceByKey`

- Return a new RDD by combining the values with the same key with a given function

- Program:

```
var a = Array(('a',3), ('b',4), ('c',5), ('a',4), ('b',-6) )  
var x = sc.parallelize(a)  
var y = x.reduceByKey( (x,y) => x+y )  
x.collect().foreach(println)  
y.collect().foreach(println)
```

Sum ints of tuples with  
same key

- Source-RDD x: ('a',3), ('b',4), ('c',5), ('a',4), ('b',-6)
- Reduce-RDD y: ('a',7), ('b',-2), ('c',5)

# Example: transformation `filter`

- Return a new RDD that only has elements that pass the `filter()` function

- Program:

```
var a = Array(1,2,3,4,5)
var x = sc.parallelize( a )
var y = x.filter(z => z % 2 != 0)
x.collect().foreach(println)
y.collect().foreach(println)
```

Remove even  
entries

- Source-RDD x: [1, 2, 3, 4, 5]

- Filter-RDD y: [1, 3, 5]

- The algorithm has also been discussed in the chapter about MapReduce
- Steps
  1. Initialize each page's rank to 1.0
  2. On each iteration, have page  $p$  send a contribution of  $\text{rank}(p) / \text{numNeighbors}(p)$  to its neighbors (the pages it has links to).
  3. Set each page's rank to  $0.15 + 0.85 * \text{contributionsReceived}$   
The last two steps repeat for several iterations, during which the algorithm will converge to the correct PageRank value for each page.
    - In practice, it's typical to run about 10 iterations.

# Example: Page Rank

## ■ Step 1: Initialize each page's rank to 1.0

```
import org.apache.spark.HashPartitioner
```

```
val list = List( ("MapR",      List("Baidu","Blogger")),  
                 ("Baidu",    List("MapR")),  
                 ("Blogger",  List("Google","Baidu")),  
                 ("Google",   List("MapR"))  
               )
```

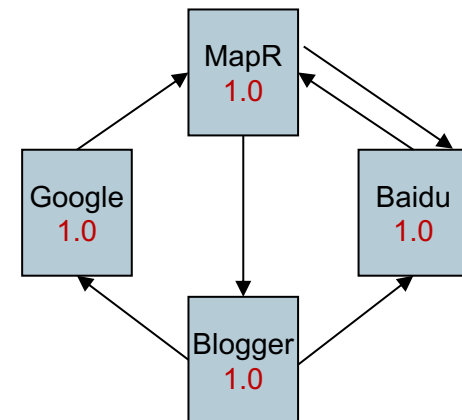
Data type of outbound links is  
(String, List(String))  
= PairRDD (url, List of links)

```
val links = sc.parallelize(list).persist()  
val ranks = links.mapValues( v => 1.0 )
```

For key-value tuples.  
Create rank RDD and set  
value for each key to 1.0

Keep RDDs in memory

```
scala> ranks.collect().foreach(println)  
(MapR,1.0)  
(Baidu,1.0)  
(Blogger,1.0)  
(Google,1.0)
```



# Example Page Rank

- Step 2: On each iteration, have page  $p$  send a contribution of  $\text{rank}(p) / \text{numNeighbors}(p)$  to its neighbors (the pages it has links to)

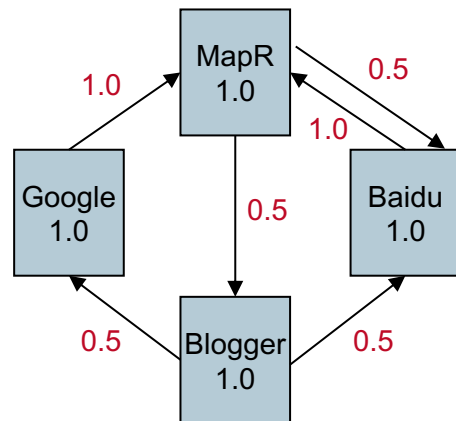
Join RDDs `links` and `ranks` using  
key=url  $\rightarrow$  resulting data type  
(String, List(String), Double)

```
val contributions = links.join(ranks).flatMap {  
  case (url, (links, rank)) =>  
    links.map(dest => (dest, rank / links.size))  
}
```

Deconstruct object \*)

```
scala> contributions.collect().foreach(println)  
(MapR,1.0)  
(Baidu,0.5)  
(Blogger,0.5)  
(Google,0.5)  
(Baidu,0.5)  
(MapR,1.0)
```

Calc contribution for each entry  
in List (destination url) and  
output tuple (url, rank)



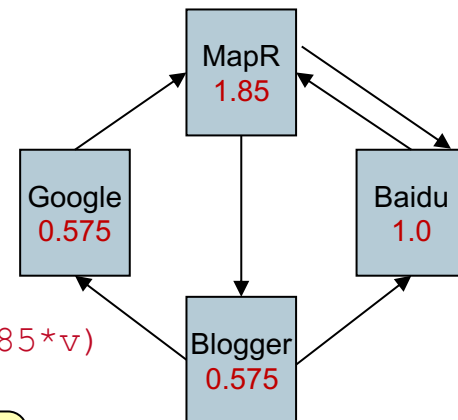
# Example Page Rank

- Step 3: Set each page's rank to  
 $0.15 + 0.85 * \text{contributionsReceived}$

```
val ranks = contributions.reduceByKey(  
    (x, y) => x + y ).mapValues(v => 0.15 + 0.85*v)
```

Sum up the contributions  
received for each key

$0.15 + 0.85 * \text{contributionsReceived}$



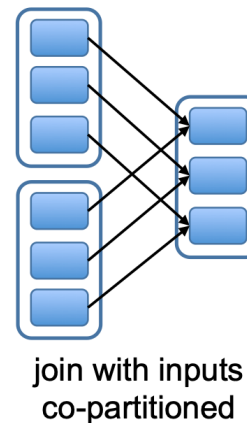
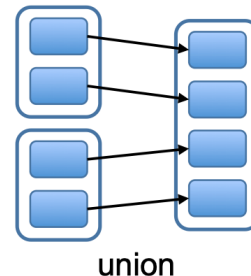
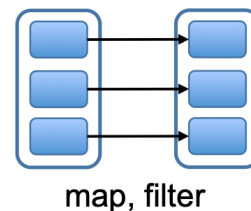
```
scala> ranks.collect().foreach(println)  
(Google,0.575)  
(MapR,1.8499999999999999)  
(Blogger,0.575)  
(Baidu,1.0)
```

- Motivation
- Overview
- Spark Programming Interface
- Representing RDDs
- Implementation

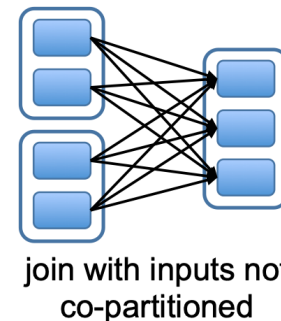
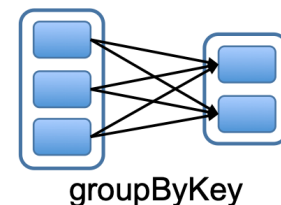
- RDDs use a simple graph-based representation for tracking lineage
- Information stored with each RDD:
  - **Partitions**: atomic pieces of the dataset
  - **Dependencies**: parent RDDs
  - **Function**: the operation used to compute the RDD based on its parents
  - **Additional metadata**: partitioning scheme and data placement
    - For example, an RDD representing an HDFS file has a partition for each block of the file and knows which machines each block is on.

- Dependencies between parent and child(ren) are classified into two types
- **Narrow dependency:** the partition of the parent RDD is used by at most one partition of the child RDD
- **Wide dependency:** multiple child partitions depend on one parent RDD partition

Narrow Dependencies:



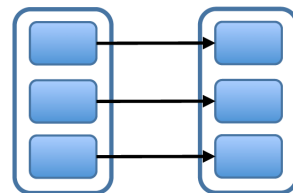
Wide Dependencies:



- Dependencies between parent and child(ren) are classified into two types
- **Narrow dependency** (more efficient)
  - Allow for pipelined execution on one cluster node
    - For example, one can apply a map followed by a filter on an element-by-element basis.
  - Recovery after a node failure is fast
    - Only the lost parent partitions need to be recomputed (can be recomputed in parallel on different nodes)
- **Wide dependency** (slower)
  - Require data from all parent partitions to be available and to be shuffled across the nodes
  - Recovery can be slow
    - A single failed node might cause the loss of some partition, requiring a complete re-execution for all partitions (checkpoint help to avoid this situation)

## ■ map:

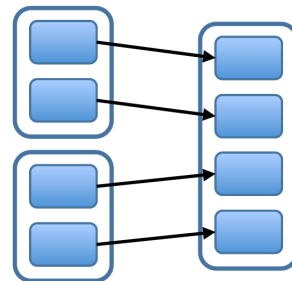
- The MappedRDD object has the same partitions and preferred locations as its parent, but applies the function passed to map to the parent's records in its iterator method.



map, filter

## ■ union:

- Calling union on two RDDs returns an RDD whose partitions are the union of those of the parents RDDs
- Each child partition is computed through a narrow dependency on the corresponding parent



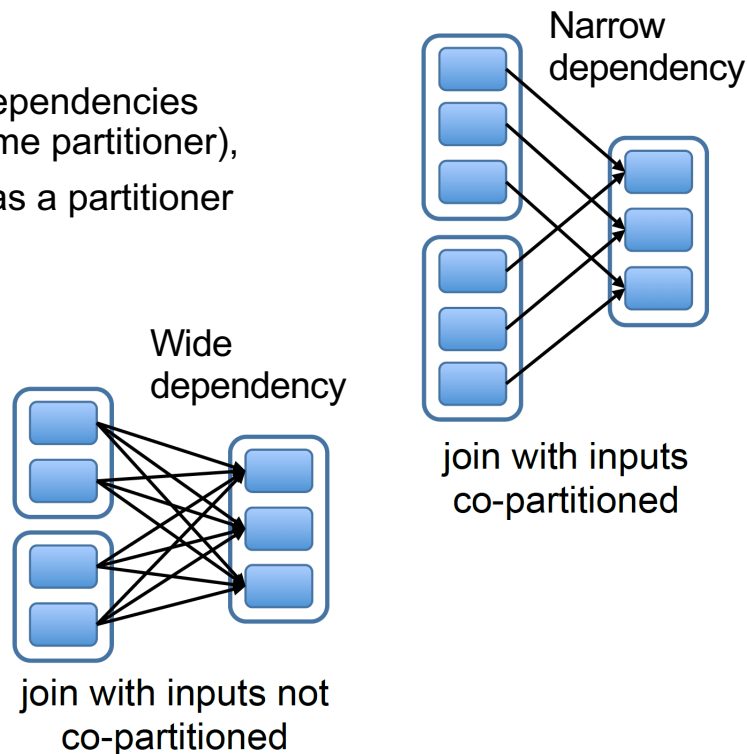
union

## ■ join:

- Joining two RDDs may lead to either two narrow dependencies (if they are both hash/range partitioned with the same partitioner),
- Or to wide dependencies, or a mix (if one parent has a partitioner and one does not).
- In either case, the output RDD has a partitioner (either one inherited from the parents or a default hash partitioner).

## ■ Remarks

- RDD's are partitioned
- Co-partitioned RDD's uses same partitioner and thus have their data distributed across partitions in same manner.



- We can optimize communication in PageRank by controlling the partitioning of the RDDs
- Approach: specify the same partitioning for links and ranks
  - ensures that the join operation between links and ranks requires no communication
    - Because of this co-partitioning each URL's rank will be on the same machine as its link list
- We can also write a custom Partitioner class to group pages that link to each other together (e.g., partition the URLs by domain name).
- Both optimizations can be expressed by calling `partitionBy` when we define links

# Example Page Rank - Optimizing RDD placement

- Co-Partitioning is done using `partitionBy`, typically before `join`
  - Requires Pair RDDs (key, value)
    - In our example the keys are string literals for urls
  - Here the `HashPartitioner` creates 4 partitions
  - The `ranks` RDD inherits the partitioner from its parent RDD `links`
    - `ranks` are also Pair RDDs with url as key

```
val links = sc.parallelize(list)
                .partitionBy(new HashPartitioner(4))
                .persist()
val ranks = links.mapValues( v => 1.0 )
```

hash-based partitioning using  
Java's `Object.hashCode`

# Example Page Rank - Performance

Input data: 54 GB Wikipedia dump, 4 million articles

